Tutorial 1 – Implement Bezier Curve

What is a Bezier Curve?

Bezier curves are an important concept in mathematical approximation of natural geometric shapes as they are known for their flexibly and precision in representing curves. Also, it is commonly used in animation engines, offering a way to provide smooth transitions with a set of control points. There are many types of Bezier curve such as Quadratic and Cubic Bezier.

The parametric equation of a Bezier curve can be written as:

$$B(t) = \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i} t^i P_i$$

Unity Project

First, create a script folder and create a new C# script with the name 'BezierCurve'. Afterwards, remove Monobehaviour as we don't want it to derive the start and update method.

Then, declare a function that returns an interpolated measure Point given T where T can be between 0 and 1 which are both inclusive. This function will correspond to the Bezier curve.

```
public static Vector3 Point3(float t, List<Vector3> controlPoints)
```

NOTE: The function we made is static as we don't have to instantiate a Bezier Curve class to get a Bezier point given a set of control points.

Now, we implement the actual calculation of the Bezier point. Therefore, we'll analyse the function below.

$$\mathbf{B}(t) = \sum_{i=0}^{n} b_{i,n}(t)\mathbf{P}_i, \quad 0 \le t \le 1$$

Where

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}$$

These three diagram don't seem too hard to implement. Now, we will calculate the Binomial coefficient with inputs n and I, both integer values.

Bernstein Basis Polynomials

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

Defined as a sequence of polynomials that converge uniformly to a continuous function on a given intervals which also retains the convexity of the function.

We'll have to calculate the values of:

- Factorial of n
- Factorial of i and
- Factorial of (n – i)

The function called Factorial will be implemented which calculates the factorial value give an integer input. Fortunately, the Factorial values of numbers can be precalculated up to a maximum value of n (18 or 20). Therefore, any Bezier curves that have a maximum value or less will be created into a Bezier Curve.

```
public class BezierCurve
{
    private static float[] Flactorial = new float[]
    {
        // Create a lookup table of the flactorial values
        // to improve performance instead of calculating
        // Set the values up to 18

        1.0f,
        1.0f,
        2.0f,
        6.0f,
        24.0f,
        120.0f,
        720.0f,
        5040.0f,
        40320.0f,
        362880.0f,
        3628800.0f,
        39916800.0f,
        479001600.0f,
        6227020800.0f,
        87178291200.0f,
        1307674368000.0f,
        20922789888000.0f

    };
```

After adding the look up table, we can continue further with the implementation of the Binomial coefficient

```
// Create the Binomial function based on the Bionomial coefficient formula
1 reference
private static float Binomial(int n, int i)
{
    float ni;
    float a1 = Flactorial[n];
    float a2 = Flactorial[i];
    float a3 = Flactorial[n - i];
    ni = a1 / (a2 * a3);
    return ni;
}
```

NOTE: The function is private as it'll only allow internal access to this function.

Next, we'll calculate the Berstein basis polynomials:

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

Afterwards, we will calculate the Binomial coefficient and the two power terms. As we've already calculated the Binomial coefficient above. The function below shows the implementation of the calculation of Bernstein basis polynomials.

```
// Afterwards, create the Bernstein basis points function
4 references
private static float Bernstein(int n, int i, float t)
{
    float t_i = Mathf.Pow(t, i);
    float t_n_i = Mathf.Pow(1 - t, n - i);

    float basis = Binomial(n, i) * t * t_i * t_n_i;
    return basis;
}
```

Finally, we'll calculate the Bezier point:

$$\mathbf{B}(t) = \sum_{i=0}^{n} b_{i,n}(t)\mathbf{P}_i, \quad 0 \le t \le 1$$

```
// Then, impelemnt the Bezier curve point function
// This function will return the interpolated poinmt where t must be between 0 and 1
0 references
public static Vector3 Point3(float t, List<Vector3> controlPoints)
{
    // This functyion will be used to return a bezier curve point which is based on the Bezier curve equation
    // t can be between 0 and 1 which is both inclusive
    int N = controlPoints.Count - 1; // Degree of the curve
    if (N > 18)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        // It'll remove the extra control points more than 18 counts
        controlPoints.RemoveRange(18, controlPoints.Count - 18);
    }

    if (t <= 0) return controlPoints[0];
    if (t >= 1) return controlPoints[controlPoints.Count - 1];

    Vector3 p = new Vector3();
    for (int i = 0; i < controlPoints.Count; i++)
    {
        Vector3 bn = Bernstein(N, i, t) * controlPoints[i];
        p += bn;
    }
    return p;
}
```

Successfully we have implemented the calculation of a Bezier point for a given set of control points at an interval t.

To get the whole list of Bezier points that represents the Bezier curve then we'll need to calculate the Bezier points for the entire point from t = n (where the point is the same at the start control point) to t = 1 (where the point is the same as the end control point). The interval duration will depend on the problem and the distance of the points.

Afterwards, lets implement the method that returns the list of points representing the Bezier curve

```
// Now calculate all the points for the entire curve from t = 0 to t = 1 with an increment of 0.01
0 references
public static List<Vector3> PointsList3(List<Vector3> controlPoints, float interval = 0.01f)
{
    int N = controlPoints.Count - 1;
    if (N > 18)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        controlPoints.RemoveRange(18, controlPoints.Count - 18);
    }

    List<Vector3> points = new List<Vector3>();
    for (float t = 0.0f; t <= 1.0f + interval - 0.0001f; t += interval)
    {
        Vector3 p = new Vector3();
        for (int i = 0; i < controlPoints.Count; ++i)
        {
            Vector3 bn = Bernstein(N, i, t) * controlPoints[i];
            p += bn;
        }
        points.Add(p);
    }
    return points;
}
```

Then, implement the same for Vector2, as shown below

```
public static Vector2 Point2(float t, List<Vector2> controlPoints)
{
    int N = controlPoints.Count - 1;
    if (N > 16)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        controlPoints.RemoveRange(18, controlPoints.Count - 16);
    }

    if (t <= 0) return controlPoints[0];
    if (t >= 1) return controlPoints[controlPoints.Count - 1];

    Vector2 p = new Vector2();
    for (int i = 0; i < controlPoints.Count; i++)
    {
        float bn = Bernstein(N, i, t);
        p += bn * controlPoints[i];
    }
    return p;
}

3 references
public static List<Vector2> PointsList2(List<Vector2> controlPoints, float interval = 0.01f)
{
    int N = controlPoints.Count - 1;
    if (N > 16)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        controlPoints.RemoveRange(18, controlPoints.Count - 16);
    }

    List<Vector2> points = new List<Vector2>();
    for (float t = 0.0f; t <= 1.0f + interval - 0.0001f; t += interval)
    {
        Vector2 p = new Vector2();
        for (int i = 0; i < controlPoints.Count; ++i)
        {
            float bn = Bernstein(N, i, t);
            p += bn * controlPoints[i];
        }
        points.Add(p);
    }
    return points;
}
```

This concludes the implementation of the Bezier Curve class.

Testing Bezier Curves

This section we will create a prefab that will use to display our control points on the scene.

First, right- click and add a new Circle Sprite.

It'll be used to represent the control points and rename the sprite to Point. Then click the add component from the Inspector which will prompt you to make a new script and call it Point_Viz.

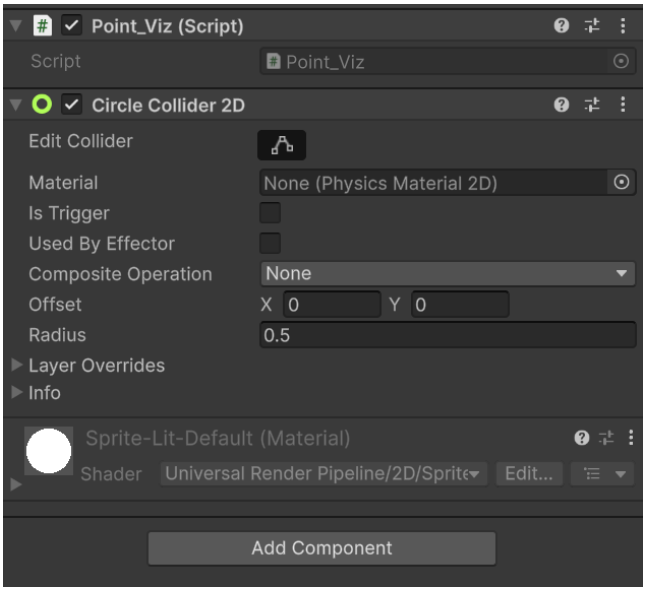
Add a private variable named mOffset.

```
private Vector3 nOffset = Vector3.zero;
```

Add OnMouseDown, OnMouseDrag and OnMouseUp methods and implement the necessary code to use mouse click to select the sprite and drag to move it.

```
    private void OnMouseDown()
    {
        if(EventSystem.current.IsPointerOverGameObject())
        {
            return;
        }

        nOffset = transform.position - Camera.main.ScreenToWorldPoint(
            new Vector3(Input.mousePosition.x,Input.mousePosition.y, 0.0f));
    }

    Unity Message | 0 references
    private void OnMouseDrag()
    {
        if(EventSystem.current.IsPointerOverGameObject())
        {
            return;
        }

        Vector3 curScreenPoint = new Vector3(Input.mousePosition.x, Input.mousePosition.y, 0.0f);
        Vector3 curPosition = Camera.main.ScreenToWorldPoint(curScreenPoint) + nOffset;
        transform.position = curPosition;
    }

    Unity Message | 0 references
    private void OnMouseUp()
    {
        if(EventSystem.current.IsPointerOverGameObject())
        {
            return;
        }
    }
}
```

Afterwards, we'll add a collider to the sprite. Select the sprite and add a new component called Circle Collider 2D.



Bezier Curve Visualization

Create a new Empty Game Object and rename the empty object to Bezier as it'll be displaying the control points and Bezier curve.

Then create a script and call it Bezier

```
public List<Vector2> controlPoints = new List<Vector2>
```

Next, we'll use a public variable that will hold the Point Prefab as it'll create instances of this prefab to display our control points.

```
// Reference to the point perfab
// as it represents each control point
public GameObject PointPrefab;
```

Next, we will add two Line Renderers to show the lines and curves. The first one is to show straight lines connecting the various control points. As for the second one will show the Bezier curve.

```
// Also will be using the LineRenderer component to show the straight lines
// connecting to these control points as well as the Bezier curve itself
LineRenderer[] mLineRenderers = null;
```

Then, we can have a List<GameObject> to hold the instantiated control point game objects.

```
// Keep track of the instantiated control point game objects
List<GameObject> mPointGameObjects = new List<GameObject>();
```

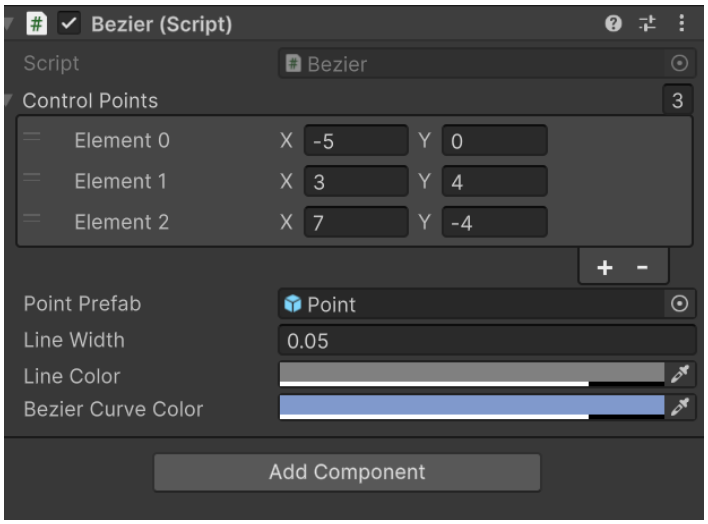
We will allow setting the line width and line colour from Unity editor.

```
// Store the properties of the line
public float LineWidth;
public float LineWidthBezier;
public Color LineColor = new Color(0.5f, 0.5f, 0.5f, 0.8f);
public Color BezierCurveColor = new Color(0.5f, 0.6f, 0.8f, 0.8f);
```

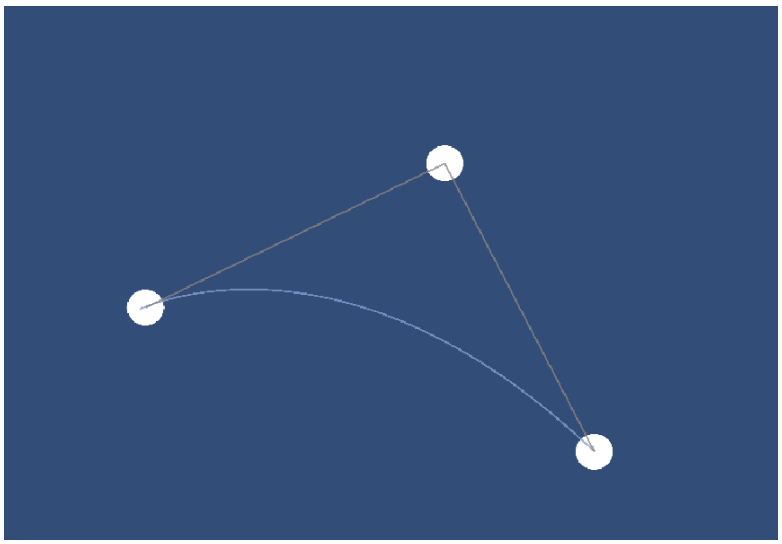
Now, were set to move onto changing some values on the Unity Editor.

Changing Values in Unity Editor

Create three control points as shown below and se the values for each of these control points. Se the Line Width to 0.05.



Click play and it should show up like this:



Final Script:

Bezier Curve:

```
using NUnit.Framework;
using System;
using System.Collections.Generic;
using UnityEngine;

3 references
public class BezierCurve
{
    private static float[] Flactorial = new float[]
    {
        // Create a lookup table of the flactorial values
        // to improve performance instead of calculating
        // Set the values up to 18

        1.0f,
        1.0f,
        2.0f,
        6.0f,
        24.0f,
        120.0f,
        720.0f,
        5040.0f,
        40320.0f,
        362880.0f,
        3628800.0f,
        39916800.0f,
        479001600.0f,
        6227020800.0f,
        87178291200.0f,
        1307674368000.0f,
        20922789888000.0f

    };

    // Create the Binomial function based on the Bionomial coefficient formula
    1 reference
    private static float Binomial(int n, int i)
    {
        float ni;
        float a1 = Flactorial[n];
        float a2 = Flactorial[i];
        float a3 = Flactorial[n - i];
        ni = a1 / (a2 * a3);
        return ni;
    }

    // Afterwards, create the Bernstein basis points function
    4 references
    private static float Bernstein(int n, int i, float t)
    {
        float t_i = Mathf.Pow(t, i);
        float t_n_i = Mathf.Pow(1 - t, n - i);

        float basis = Binomial(n, i) * t * t_i * t_n_i;
        return basis;
    }
```

```
// Then, impelemnt the Bezier curve point function
// This function will return the interpolated poinmt where t must be between 0 and 1
0 references
public static Vector3 Point3(float t, List<Vector3> controlPoints)
{
    // This functyion will be used to return a bezier curve point which is based on the Bezier curve equation
    // t can be between 0 and 1 which is both inclusive
    int N = controlPoints.Count - 1; // Degree of the curve
    if (N > 18)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        // It'll remove the extra control points more than 18 counts
        controlPoints.RemoveRange(18, controlPoints.Count - 18);
    }

    if (t <= 0) return controlPoints[0];
    if (t >= 1) return controlPoints[controlPoints.Count - 1];

    Vector3 p = new Vector3();
    for (int i = 0; i < controlPoints.Count; i++)
    {
        Vector3 bn = Bernstein(N, i, t) * controlPoints[i];
        p += bn;
    }
    return p;
}

// Now calculate all the points for the entire curve from t = 0 to t = 1 with an increment of 0.01
0 references
public static List<Vector3> PointsList3(List<Vector3> controlPoints, float interval = 0.01f)
{
    int N = controlPoints.Count - 1;
    if (N > 18)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        controlPoints.RemoveRange(18, controlPoints.Count - 18);
    }

    List<Vector3> points = new List<Vector3>();
    for (float t = 0.0f; t <= 1.0f + interval - 0.0001f; t += interval)
    {
        Vector3 p = new Vector3();
        for (int i = 0; i < controlPoints.Count; ++i)
        {
            Vector3 bn = Bernstein(N, i, t) * controlPoints[i];
            p += bn;
        }
        points.Add(p);
    }
    return points;
}
```

```
public static Vector2 Point2(float t, List<Vector2> controlPoints)
{
    int N = controlPoints.Count - 1;
    if (N > 16)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        controlPoints.RemoveRange(18, controlPoints.Count - 16);
    }

    if (t <= 0) return controlPoints[0];
    if (t >= 1) return controlPoints[controlPoints.Count - 1];

    Vector2 p = new Vector2();
    for (int i = 0; i < controlPoints.Count; i++)
    {
        float bn = Bernstein(N, i, t);
        p += bn * controlPoints[i];
    }
    return p;
}

3 references
public static List<Vector2> PointsList2(List<Vector2> controlPoints, float interval = 0.01f)
{
    int N = controlPoints.Count - 1;
    if (N > 16)
    {
        Debug.Log("You have used more than 18 control points. The maxium allowable control" + "points for this tutorial is 18");
        controlPoints.RemoveRange(18, controlPoints.Count - 16);
    }

    List<Vector2> points = new List<Vector2>();
    for (float t = 0.0f; t <= 1.0f + interval - 0.0001f; t += interval)
    {
        Vector2 p = new Vector2();
        for (int i = 0; i < controlPoints.Count; ++i)
        {
            float bn = Bernstein(N, i, t);
            p += bn * controlPoints[i];
        }
        points.Add(p);
    }
    return points;
}
```

Bezier:

```
using NUnit.Framework;
using System.Collections.Generic;
using Unity.VisualScripting;
using UnityEngine;

Unity Script (1 asset reference) | 0 references
public class Bezier : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    public List<Vector2> controlPoints = new List<Vector2>
    {
        new Vector2(-5.0f, -5.0f),
        new Vector2(0.0f, 2.0f),
        new Vector2(5.0f, -2.0f)
    };

    // Reference to the point perfab
    // as it represents each control point
    public GameObject PointPrefab;

    // Also will be using the LineRenderer component to show the straight lines
    // connecting to these control points as well as the Bezier curve itself
    LineRenderer[] mLineRenderers = null;

    // Keep track of the instantiated control point game objects
    List<GameObject> mPointGameObjects = new List<GameObject>();

    // Store the properties of the line
    public float LineWidth;
    public float LineWidthBezier;
    public Color LineColor = new Color(0.5f, 0.5f, 0.5f, 0.8f);
    public Color BezierCurveColor = new Color(0.5f, 0.6f, 0.8f, 0.8f);

    // Create a function to create a line renderer
    2 references
    private LineRenderer CreateLine()
    {
        GameObject obj = new GameObject();
        LineRenderer lr = obj.AddComponent<LineRenderer>();
        lr.material = new Material(Shader.Find("Sprites/Default"));
        lr.startColor = LineColor;
        lr.endColor = LineColor;
        lr.startWidth = LineWidth;
        lr.endWidth = LineWidth;
        return lr;

    }
    Unity Message | 0 references
```

```
private void Start()
{
    // Create the actual lines
    mLineRenderers = new LineRenderer[2];
    mLineRenderers[0] = CreateLine();
    mLineRenderers[1] = CreateLine();

    // Set the name of the lines to be distingushed in the hierarchy
    mLineRenderers[0].gameObject.name = "LineRenderer_obj_0";
    mLineRenderers[1].gameObject.name = "LineRenderer_obj_1";

    // Create the instances of the control points
    for (int i = 0; i < controlPoints.Count; i ++)
    {
        GameObject obj = Instantiate(PointPrefab, controlPoints[i], Quaternion.identity);
        obj.name = "ControlPoint_" + i.ToString();
        mPointGameObjects.Add(obj);
    }

}

Unity Message | 0 references
private void Update()
{
    LineRenderer lineRenderer = mLineRenderers[0];
    LineRenderer curveRenderer = mLineRenderers[1];

    List<Vector2> pts = new List <Vector2>();
    for (int i = 0; i < mPointGameObjects.Count; i ++)
    {
        pts.Add(mPointGameObjects[i].transform.position);
    }

    lineRenderer.positionCount = pts.Count;
    for(int i = 0; i < pts.Count; i ++)
    {
        lineRenderer.SetPosition(i, pts[i]);
    }

    List<Vector2> curve = BezierCurve.PointsList2(pts, 0.01f);
    curveRenderer.startColor = BezierCurveColor;
    curveRenderer.endColor = BezierCurveColor;
    curveRenderer.positionCount = curve.Count;
    curveRenderer.startWidth = LineWidthBezier;

    for(int i = 0; i < curve.Count; i ++)
    {
        curveRenderer.SetPosition(i, curve[i]);
    }
}
```

Point:

```
using System.Collections.Specialized;
using UnityEngine;
using UnityEngine.EventSystems;

Unity Script (1 asset reference) | 0 references
public class Point_Viz : MonoBehaviour
{
    private Vector3 nOffset = Vector3.zero;
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    Unity Message | 0 references
    void Start()
    {

    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }

    Unity Message | 0 references
    private void OnMouseDown()
    {
        if(EventSystem.current.IsPointerOverGameObject())
        {
            return;
        }

        nOffset = transform.position - Camera.main.ScreenToWorldPoint(
            new Vector3(Input.mousePosition.x,Input.mousePosition.y, 0.0f));
    }

    Unity Message | 0 references
    private void OnMouseDrag()
    {
        if(EventSystem.current.IsPointerOverGameObject())
        {
            return;
        }

        Vector3 curScreenPoint = new Vector3(Input.mousePosition.x, Input.mousePosition.y, 0.0f);
        Vector3 curPosition = Camera.main.ScreenToWorldPoint(curScreenPoint) + nOffset;
        transform.position = curPosition;
    }

    Unity Message | 0 references
    private void OnMouseUp()
    {
        if(EventSystem.current.IsPointerOverGameObject())
        {
            return;
        }
    }
}
```